

DIA 5 — Como Conectar Tabelas sem Duplicar Informações

Introdução

Um dos maiores erros cometidos por quem está começando a trabalhar com bancos de dados é duplicar informações desnecessariamente.

À primeira vista, parece fazer sentido armazenar todos os dados juntos. Afinal, se um pedido pertence a um cliente, por que não gravar todas as informações do cliente dentro da própria tabela de pedidos?

Embora essa abordagem pareça simples, ela cria diversos problemas de manutenção, aumenta o espaço ocupado pelo banco de dados e compromete a integridade das informações.

Neste capítulo vamos entender por que os bancos de dados relacionais foram criados dessa forma e como as **chaves estrangeiras** e os **JOINS** resolvem esse problema.

O Problema da Duplicação de Dados

Imagine que sua empresa possui **500 mil pedidos** cadastrados.

Se, para cada pedido, você armazenar novamente todas as informações do cliente, sua tabela ficará parecida com esta:

PEDIDOS

ID	Cliente	Telefone	Cidade
----	---------	----------	--------

Agora imagine que esse cliente troca de telefone.

O que acontece?

Todos os pedidos antigos continuam armazenando o telefone antigo.

Pedido 1 → (48) 99999-1111

Pedido 2 → (48) 99999-1111

Pedido 3 → (48) 99999-1111

Pedido 4 → (48) 99999-1111

...

Se esse cliente realizou milhares de compras ao longo dos anos, você precisará atualizar milhares de registros.

Além de ser uma operação extremamente custosa, ela aumenta muito a chance de inconsistências.

Imagine que apenas alguns registros sejam atualizados.

Qual telefone passa a ser o correto?

O banco de dados deixa de representar corretamente a realidade.

Esse problema recebe o nome de **redundância de dados**.

Um Modelo Incorreto

Uma estrutura como a abaixo parece simples, mas apresenta diversos problemas.

PEDIDOS

id
cliente_nome
cliente_cpf
cliente_email
cliente_telefone
cliente_cidade
cliente_estado
produto
valor

Agora pense em um cliente chamado João.

Sempre que ele realizar uma compra, todas as suas informações serão gravadas novamente.

João
João
João
João
João
João

Nome.

CPF.

Telefone.

Cidade.

Estado.

Tudo será repetido centenas ou milhares de vezes.

Estamos armazenando exatamente a mesma informação inúmeras vezes.

Além do desperdício de espaço, qualquer alteração futura exigirá modificar todos esses registros.

Esse é justamente um dos principais problemas que a modelagem relacional procura resolver.

A Solução: Separar as Informações

Em vez de armazenar todas as informações do cliente dentro da tabela de pedidos, criamos duas tabelas independentes.

Uma tabela para clientes.

CLIENTES

id
nome
telefone
email

E outra para pedidos.

PEDIDOS

id
cliente_id
valor
data

Observe que agora o pedido não precisa conhecer o telefone do cliente.

Ele não precisa saber o e-mail.

Nem a cidade.

Nem o CPF.

Tudo o que o pedido precisa saber é **quem realizou aquela compra**.

Para isso basta armazenar um único número.

cliente_id = 15

Esse número identifica exatamente qual cliente pertence àquele pedido.

Todas as demais informações permanecem armazenadas em apenas um lugar.

Caso o cliente altere seu telefone, basta atualizar um único registro.

Todos os pedidos automaticamente passarão a exibir o novo telefone sempre que consultados.

Essa é uma das maiores vantagens dos bancos de dados relacionais.

Chave Estrangeira (Foreign Key)

Agora que entendemos o problema, podemos apresentar o conceito formal.

Considere as seguintes tabelas.

CLIENTES

id
nome

PEDIDOS

id
cliente_id

A coluna `cliente_id` não armazena informações do cliente.

Ela apenas referencia outro registro.

Dizemos que essa coluna é uma **chave estrangeira**, ou **Foreign Key**.

Sua função é ligar duas tabelas.

Em outras palavras, a chave estrangeira cria um relacionamento entre os registros.

Ela garante que um pedido sempre esteja associado a um cliente válido, evitando referências inexistentes.

Além disso, o próprio banco de dados passa a garantir a integridade dessas informações.

JOIN: Conectando as Tabelas

Se as informações estão separadas, surge uma pergunta natural.

Como recuperar tudo junto novamente?

A resposta está nas cláusulas **JOIN**.

Os JOINS permitem combinar informações provenientes de duas ou mais tabelas em um único resultado.

Entre os diversos tipos existentes, o mais utilizado é o **INNER JOIN**.

Ele retorna apenas os registros que possuem correspondência nas duas tabelas.

Vamos utilizar o seguinte exemplo.

Criando as tabelas

```
CREATE TABLE clientes (  
    id BIGSERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    cpf CHAR(11) NOT NULL UNIQUE,  
    email VARCHAR(150),  
    telefone VARCHAR(20),  
    cidade VARCHAR(100),  
    estado CHAR(2)  
);  
  
CREATE TABLE pedidos (  
    id BIGSERIAL PRIMARY KEY,  
    cliente_id BIGINT NOT NULL,  
    data_pedido DATE NOT NULL,  
    valor NUMERIC(10,2) NOT NULL,  
  
    CONSTRAINT fk_pedidos_clientes  
        FOREIGN KEY (cliente_id)  
        REFERENCES clientes(id)  
);
```

Observe que a coluna `cliente_id` da tabela `pedidos` é uma chave estrangeira que aponta para a tabela `clientes`.

Inserindo dados

Agora vamos cadastrar alguns clientes.

```
INSERT INTO clientes
(nome, cpf, email, telefone, cidade, estado)
VALUES
('João Silva', '11111111111', 'joao@email.com', '(48)99999-1111',
'Criciúma', 'SC'),
('Maria Souza', '22222222222', 'maria@email.com', '(48)99999-2222',
'Florianópolis', 'SC'),
('Carlos Oliveira', '33333333333', 'carlos@email.com', '(51)99999-3333',
'Porto Alegre', 'RS');
```

Depois cadastramos alguns pedidos.

```
INSERT INTO pedidos
(cliente_id, data_pedido, valor)
VALUES
(1, '2026-06-01', 150.00),
(1, '2026-06-03', 320.50),
(1, '2026-06-08', 80.90),

(2, '2026-06-02', 920.00),
(2, '2026-06-04', 110.30),

(3, '2026-06-05', 75.00),
(3, '2026-06-09', 410.00);
```

Perceba que a tabela `pedidos` armazena apenas o identificador do cliente.

Nenhuma informação foi duplicada.

Recuperando as Informações

Mesmo com os dados separados, podemos exibir tudo em uma única consulta utilizando o `INNER JOIN`.

```
SELECT
    p.id AS pedido,
    c.nome,
    c.telefone,
    p.data_pedido,
    p.valor
FROM pedidos p
INNER JOIN clientes c
    ON c.id = p.cliente_id
ORDER BY p.id;
```

Observe o que acontece.

Embora a tabela `pedidos` não possua as colunas `nome` e `telefone`, o banco consegue recuperá-las automaticamente.

Isso acontece porque o SQL utiliza a condição:

```
ON c.id = p.cliente_id
```

para localizar o cliente correspondente a cada pedido.

O resultado é uma única tabela contendo informações das duas tabelas ao mesmo tempo.

É exatamente isso que torna os bancos relacionais tão poderosos.

Atualizando Apenas um Registro

Agora imagine que João trocou de telefone.

Precisamos alterar apenas um registro.

```
UPDATE clientes
SET telefone = '(48)98888-0000'
WHERE id = 1;
```

Nenhum pedido precisou ser atualizado.

Ao executar novamente o `JOIN`:

```
SELECT
  p.id,
  c.nome,
  c.telefone,
  p.valor
FROM pedidos p
INNER JOIN clientes c
  ON c.id = p.cliente_id;
```

Todos os pedidos do João já aparecerão automaticamente com o novo telefone.

Não foi necessário modificar centenas ou milhares de registros.

O SQL fez toda a conexão entre as tabelas de forma transparente.

Por Que Isso é Tão Importante?

Separar informações em tabelas diferentes traz diversas vantagens.

- Evita duplicação de dados.
- Reduz o espaço ocupado pelo banco de dados.
- Facilita a manutenção das informações.
- Garante maior consistência dos dados.
- Diminui a possibilidade de erros.
- Torna as atualizações muito mais simples.

Esses princípios fazem parte do processo conhecido como **normalização de banco de dados**, um dos fundamentos dos bancos relacionais.

Conclusão

Neste capítulo entendemos por que não devemos duplicar informações em tabelas diferentes.

Vimos que armazenar repetidamente os dados de um cliente gera desperdício de espaço, dificulta atualizações e aumenta a chance de inconsistências.

Aprendemos então como utilizar **chaves estrangeiras (Foreign Keys)** para conectar tabelas e descobrimos que o **INNER JOIN** permite recuperar todas essas informações automaticamente durante uma consulta.

A partir deste ponto, você começa a enxergar um banco de dados da forma como ele realmente funciona: um conjunto de tabelas relacionadas entre si, onde cada informação é armazenada apenas uma vez e reutilizada sempre que necessário.